A+ Plane

Backend Vibe-Coding Prompt Book

A professional English prompt collection for AI-assisted development of the A+ Plane backend, APIs, authentication, database integration, and testing.

Project	A+ Plane — College Grievance and Campus Support Portal
Backend stack	Node.js, Express, TypeScript, Prisma ORM, PostgreSQL on Neon, JWT, bcryptjs, Zod
Development style	Iterative AI-assisted / vibe coding: prompt, inspect, generate or refine code, run commands, debug observed errors, and verify behavior.
Purpose	Document prompts for backend setup, data models, authentication, complaint workflows, admin APIs, AI assistant integration, security, and deployment readiness.

Documentation note: This is a reusable prompt book based on the A+ Plane backend requirements and setup details discussed for the project. It is not a verbatim record of every previous chat, and it does not prove that every listed feature has been implemented, tested, or deployed.

1. How to Use This Prompt Book

Open the existing A+ Plane project in your AI coding environment. Use the master prompt first, then run the focused prompts in the order that matches the current state of the repository. The assistant should inspect existing files and preserve working code and data before making changes.

Recommended workflow

- **Inspect:** read the existing backend folder, package.json, Prisma schema, routes, middleware, services, and environment configuration.
- **Plan:** identify the next small change and its dependencies.
- **Implement:** edit the existing files and create only missing files that are necessary.
- **Run:** execute the relevant install, validation, migration, development, lint, test, or build commands.
- **Debug:** use actual terminal output and request-specific fixes.
- **Verify:** test normal, invalid, unauthorized, and error scenarios.
- **Document:** retain the prompt, changed-file list, commands, outputs, and remaining blockers.

Backend structure discussed

```
backend/
  prisma/
    schema.prisma
  src/
    middleware/
      authenticate.ts
      authorize.ts
      validate.ts
    routes/
      auth.routes.ts
      student.routes.ts
      complaint.routes.ts
      admin.routes.ts
      ai.routes.ts
    services/
      complaint.service.ts
      routing.service.ts
      ai.service.ts
    server.ts
  .env
  package.json
```

This is the structure discussed for A+ Plane, not a guarantee that every file currently exists. The coding assistant must confirm the live workspace before editing.

Environment and tooling context

The project has been discussed with Prisma 6.19.0, TypeScript, Express, and a Neon PostgreSQL database. The schema uses the prisma-client-js generator and PostgreSQL datasource. Always verify actual installed versions and package scripts before running version-specific commands.

2. Master Prompt — Build and Integrate the Backend

```
You are a senior Node.js, Express, TypeScript, Prisma, and PostgreSQL engineer working with me on A+ Plane, a college griev

FIRST: INSPECT THE REAL PROJECT
1. Read backend/package.json, prisma/schema.prisma, server.ts, routes, middleware, services, and environment configuration.
2. Check the installed Node.js, Prisma CLI, and @prisma/client versions before suggesting commands.
3. Inspect the existing frontend API service and components so the backend contract matches the current application.
4. Reuse existing working code. Do not create duplicate servers, routers, auth systems, or Prisma clients.
5. Identify missing files or endpoints explicitly before adding them.

EXPECTED STACK
- Node.js and Express
- TypeScript
- Prisma ORM
- PostgreSQL hosted on Neon
- bcryptjs for password hashing
- JWT for API authentication, using a strong environment-provided secret
- Zod for request validation
- Existing CORS, Helmet, rate limiting, and error-handling conventions where configured

PRODUCT REQUIREMENTS
- Student authentication using the college-issued Student ID and password.
- Separate admin authorization, with no public ability to create an admin account.
- Student profile data for name, Student ID, course, semester, and section where stored.
- Complaint submission and tracking, including unique ticket references and valid status transitions.
- Hostel, Subjects, Bus, Her Voice, and Lost & Found grievance categories where supported by the current application.
- Student endpoints that return only the authenticated student's own records.
- Admin endpoints that allow only authorized administrative actions.
- AI Campus Assistant endpoints that use real, permitted data and handle provider unavailability honestly.
- Health-check endpoints and safe error responses.

ENGINEERING RULES
- Keep secrets in environment variables; never print or commit .env values.
- Never reset, drop, or delete the existing database to solve a migration problem.
- Do not invent routes, schema fields, provider capabilities, or test results.
- Validate all untrusted input on the server.
- Enforce ownership and role authorization in the backend, not only in the React UI.
- Use appropriate HTTP status codes and a consistent JSON response shape.
- Avoid returning password hashes, JWT secrets, stack traces, or unnecessary private data.
- Use transactions where a workflow requires multiple related database changes.
- Keep route handlers focused and reuse service functions where helpful.

WORK METHOD
1. Create a short plan, then proceed with implementation.
2. Make focused, reviewable changes and preserve existing data.
3. Run Prisma validation and appropriate migration commands only after reviewing the schema and environment.
4. Run available TypeScript, lint, test, and build commands.
5. Report actual files changed, commands run, test outcomes, and unresolved blockers.
6. Never claim the database, AI provider, tests, or deployment works unless you verified it.
```

3. Prompt — Prisma Schema and Neon PostgreSQL

```
Review and complete the existing Prisma schema and Neon PostgreSQL integration for A+ Plane.

- Inspect prisma/schema.prisma, the installed Prisma versions, package scripts, and existing migration history.
- Preserve the current database and existing records. Do not run prisma migrate reset, drop tables, or replace the database
- Confirm that the datasource uses PostgreSQL and the generator matches the installed Prisma version.
- Review the existing User/Student and Complaint relationships and align the schema with the actual backend and frontend re
- Where needed and compatible with the existing data, support:
  * Student identity: unique college Student ID, name, hashed password, course, semester, section, and role.
  * Complaint: unique ticket/reference ID, title, description, category, status, owning student, and timestamps.
  * Additional status history or assignment fields only if the product requirements and current implementation justify them
- Use enums or validated values for roles and complaint statuses where appropriate. Keep frontend status labels aligned wit
- Check required fields, unique constraints, indexes, relations, deletion behavior, and timestamp behavior.
- Use the existing DATABASE_URL environment variable for Neon. Do not hardcode connection strings or print credentials.
- Run prisma validate and inspect migration status. If a migration is needed, explain the effect and use a safe, non-destru
- Run or regenerate the Prisma client using commands compatible with the installed version.
- Report the schema changes, migration created or required, validation result, and any data-preservation risks before apply
```

4. Prompt — Student and Admin Authentication

```
Inspect and improve the existing auth.routes.ts, authenticate.ts, authorize.ts, validate.ts, and the frontend auth service

STUDENT LOGIN
- Authenticate students using their college Student ID and password only.
- Store only a strong password hash using bcryptjs; never store plaintext passwords.
- Validate input with Zod and return generic authentication errors that do not reveal whether a particular student ID exist
- Issue a signed JWT using a strong JWT_SECRET loaded from the environment. Include only the claims needed for authorizatio
- Do not return the password hash in any response.
- Use a consistent response contract that the existing frontend can consume.

ADMIN ACCESS
- Do not allow public registration to assign the ADMIN role.
- Do not trust a role supplied by the request body, query, or frontend.
- Provision an admin securely through a controlled server-side setup or protected administrative process.
- If the project requires a configured admin credential, keep it server-side in environment configuration or a secure provi
- Do not silently accept multiple hardcoded passwords.

MIDDLEWARE
- authenticate.ts must verify the token signature and expiry and safely attach the verified identity to the request.
- authorize.ts must enforce required roles server-side.
- Reject missing, invalid, expired, or malformed tokens with appropriate status codes.
- Ensure admin-only endpoints cannot be accessed by a student token.

SECURITY AND QUALITY
- Apply request validation, sensible login rate limits, secure error messages, and consistent HTTP status codes.
- Avoid logging passwords, tokens, authorization headers, or secrets.
- Verify that frontend token handling and logout behavior match the API contract.
- Test successful login, wrong password, missing fields, malformed token, expired token, and role mismatch where practical.
- Report what was actually tested and any remaining limitations.
```

5. Prompt — Student Profile and Complaint APIs

```
Implement or refine the A+ Plane student and complaint API routes using the existing Express, TypeScript, Prisma, and authe

Before coding, inspect student.routes.ts, complaint.routes.ts, complaint.service.ts, schema.prisma, authenticate.ts, and th

STUDENT API
- Provide the authenticated student's profile only from the verified token identity and database record.
- Return only fields needed by the frontend, such as name, Student ID, course, semester, and section.
- Do not accept an arbitrary student ID as proof that the requester owns that record.

COMPLAINT API
- Support creating a complaint with validated title, description, and an allowed category.
- Associate the complaint with the authenticated student on the server; do not trust a studentId supplied in the request bo
- Generate a unique ticket/reference identifier using a reliable server-side strategy.
- Provide an endpoint for the signed-in student to list their own complaints.
- Provide complaint details only when the requester is the owner or has an authorized admin role.
- Use the statuses actually defined by the schema. Validate status changes and keep timestamps accurate.
- Return a consistent response format compatible with the existing React components.
- Use pagination or sensible limits if lists may grow.
- Return proper 400/401/403/404/409/500 responses as appropriate without exposing stack traces.

DATA INTEGRITY
- Use Prisma safely and handle unique-constraint and validation errors.
- Use a transaction if an operation updates multiple related records that must succeed together.
- Do not invent complaint history, ticket IDs, or status changes.
- Do not allow a student to modify another student's complaint by changing a URL ID.
- If status-history or assignment functionality is not represented in the existing schema, state that clearly and propose a

TESTS
Test or provide test cases for valid submission, invalid payload, missing authentication, duplicate submission where applic
Report the actual endpoints, request/response contracts, and test results.
```

6. Prompt — Admin Complaint Management APIs

```
Review and implement the A+ Plane admin API only to the extent supported by the product requirements and current schema.

- Inspect admin.routes.ts, authorize.ts, complaint.service.ts, schema.prisma, and the frontend AdminDashboard.jsx.
- Require a verified ADMIN role for every admin-only endpoint.
- Provide authorized complaint listing and filtering, with reasonable pagination where appropriate.
- Implement review, assignment, or status-change operations only if the data model and workflow support them.
- Validate all request parameters and status transitions on the server.
- Record accurate timestamps and, if the existing schema supports it, a clear status history or administrative action recor
- Return only information an authorized administrator needs to manage complaints.
- Never allow the browser to grant itself admin permissions.
- Return consistent responses and meaningful error codes for unauthorized, invalid, missing, and failed operations.
- Do not create fake analytics, fake complaints, or placeholder success responses.
- If assignment, status history, or metrics require schema changes, explain the proposed migration and preserve existing da
- Ensure the API contract matches AdminDashboard.jsx and the shared complaint/status components.
- Test student denial, admin success, invalid status, unknown complaint, and database error handling where practical.
```

7. Prompt — AI Campus Assistant Backend

```
Improve the A+ Plane AI assistant backend using the existing ai.routes.ts, ai.service.ts, authentication middleware, and fr

FIRST INSPECT
- Read the existing AI route, service, API client, environment configuration, and frontend AIChatbot.jsx.
- Identify whether a real AI provider is configured and which request/response fields are actually supported.
- Do not expose provider API keys or call a provider directly from the browser.

BEHAVIOR
- Accept a validated user message with a reasonable length limit.
- Return a consistent response that the frontend can safely render, such as a reply and optional quick links or an approved
- Validate any target path against an explicit allowlist of internal application routes. Never accept an arbitrary redirect
- Provide useful navigation guidance for raising grievances, Bus, Hostel, Subjects, Her Voice, and Lost & Found where those
- Where live campus data is needed, use authorized backend services and actual database/API results. Do not invent bus sche
- Only return a student's complaint information after verifying the student's identity and ownership.
- If the AI provider is missing, unavailable, or returns an error, use a clearly identified, safe fallback response. Do not
- Keep sensitive prompts and private records out of unnecessary logs. Apply rate limits and input validation.
- Respond supportively to sensitive grievance topics and direct users to the appropriate reporting flow. Do not promise ano
- Handle provider timeouts, malformed output, empty replies, and upstream errors without leaking internal details.
- Ensure the frontend can distinguish a normal response from a fallback or service-unavailable state if the API contract su
- Test successful response, empty input, excessive input, unauthenticated access where authentication is required, provider
- Report the actual AI provider status and what was verified.
```

8. Prompt — Bus and Other Campus Service Logic

```
Inspect the current A+ Plane requirements and backend structure, then determine which campus service data can be implemente

BUS MANAGEMENT
- Support the agreed campus-to-Haldwani bus route and the required departure-time window only if the actual project require
- Do not invent real-time locations, bus arrival times, departure schedules, or service availability.
- If the system records bus incidents, provide a validated report workflow for issues such as a late bus, a bus leaving wit
- Store incident reports only if the database schema and authorized API support them.
- If there is no verified live tracking source, label the feature as incident reporting or schedule information rather than

OTHER SERVICES
- Keep Hostel, Subjects, Her Voice, and Lost & Found categories aligned with the frontend forms and actual backend schema.
- Avoid inventing room inventory, course details, transport data, or policy text.
- Add new schema fields only when necessary; explain and validate migrations before applying them.
- Protect sensitive reports with appropriate access controls and do not claim a stronger privacy guarantee than the system

Return a concise list of data sources, endpoints, fields, validations, and missing integrations. Implement only features th
```

9. Prompt — Validation, Error Handling, and API Consistency

```
Audit the A+ Plane backend for reliable validation and consistent error handling.

- Inspect server.ts, all route files, middleware, and service functions.
- Confirm JSON parsing, CORS, Helmet, rate limiting, route registration, and centralized error handling are configured appr
- Place middleware in a deliberate order and ensure request-size limits and rate limits fit the application's needs.
- Use Zod schemas for request bodies, route parameters, and query parameters where appropriate.
- Return consistent JSON error responses with safe user-facing messages.
- Use suitable HTTP status codes and do not return stack traces, SQL errors, secrets, or internal implementation details to
- Log enough operational context for debugging without logging passwords, JWTs, authorization headers, sensitive grievance
- Handle Prisma connection failures, missing records, unique-constraint violations, invalid enum values, and upstream AI pr
- Ensure every async route handles rejected promises safely under the installed Express version and project conventions.
- Confirm the health endpoint reports service health accurately; do not report database health as successful if the databas
- Add or update tests for validation failures, missing authentication, role mismatch, not-found errors, and server errors.
- Report issues found, files changed, and the commands actually run.
```

10. Prompt — Security and Privacy Review

```
Conduct a security-focused review of the existing A+ Plane backend. Do not make destructive changes.

Review:
- Student ID/password authentication and password hashing
- JWT signing, verification, expiry, and secret management
- Role authorization for all admin routes
- Ownership checks for student profile and complaint data
- Public registration and prevention of user-supplied ADMIN roles
- Input validation and request size limits
- Login and AI endpoint rate limiting
- CORS and security headers
- Error responses and sensitive logging
- Environment file handling and secret exposure
- Database query patterns and safe Prisma usage
- AI prompt/data handling and access to private records
- Her Voice / sensitive grievance access and privacy wording

Rules:
- Do not print or copy secret values into logs, source files, test reports, or this prompt response.
- Do not claim compliance with a legal or security standard without a proper assessment.
- Do not promise anonymity, confidentiality, or encryption properties that have not been implemented and verified.
- Do not weaken authentication to make a frontend demo easier.
- Keep database records intact.
- For each issue, describe the risk, affected file, recommended fix, and whether the fix was applied.
- Run only safe tests against the configured environment and report actual results.
```

11. Prompt — Neon Database Connection and Deployment Readiness

```
Prepare the A+ Plane backend for a controlled deployment using the existing Neon PostgreSQL database.

- Inspect package.json scripts, Prisma version, schema, server startup, and environment variable names.
- Keep DATABASE_URL and JWT_SECRET in the deployment platform's secret/environment configuration. Never place them in front
- Confirm the server listens on the host-provided port where required and has a safe production start command.
- Ensure CORS permits only the intended frontend origins in production configuration.
- Ensure the health endpoint accurately checks service and database availability.
- Review Prisma client generation and migration deployment commands for compatibility with the installed Prisma version.
- Do not run destructive migration or database reset commands.
- Do not claim a deployment is complete if no deployment was actually performed.
- If required credentials or service access are unavailable, identify the exact blocker without requesting the user to past
- Provide a deployment checklist and safe commands based on the project's real package scripts.
- Run local validation/build checks that are possible and report their outputs.
```

12. Prompt — Backend Test Plan and Final Audit

```
Perform a final backend audit for A+ Plane. Inspect the actual repository and package scripts before running commands.

CHECKS
1. Prisma schema validation.
2. Migration status without resetting or deleting data.
3. TypeScript type-check or build, if configured.
4. Lint and automated tests, if configured.
5. Server startup and health endpoint.
6. Database connectivity using the configured environment without exposing credentials.
7. Student login success and invalid-password behavior.
8. Authentication middleware and token rejection.
9. Student access limited to their own complaints.
10. Admin-only endpoint authorization.
11. Complaint creation, listing, detail access, and status validation.
12. AI endpoint behavior when the provider is configured and when it is unavailable.
13. Validation and error handling for malformed or oversized requests.

For each check, record:
- Command or test case actually executed
- Expected behavior
- Actual observed result
- Pass, fail, or blocked
- Relevant file or issue

Do not invent results, fabricate API payloads, or mark unexecuted tests as passed. Do not expose credentials, private stude

Finish with:
- Files created or changed
- Endpoints implemented
- Database/schema changes and migration status
- Security and authorization notes
- Actual test/build results
- Known limitations and blockers
- Next steps for frontend integration
```

13. Backend Endpoint Inventory Template

Ask the coding assistant to fill this table from the actual source code. The paths below are examples of categories to inspect, not a claim that every endpoint already exists.

Area	What to document
Authentication	Implemented method/path, validation, response fields, token behavior
Student profile	Authenticated profile route, returned fields, ownership rules
Complaints	Create/list/detail/status routes, ticket ID, status enum, ownership checks
Admin	Admin-only listing, filtering, assignment/status actions actually implemented
AI assistant	Chat route, input/response schema, provider/fallback status, rate limits
Health	Health endpoint and whether it checks the database

Have the AI fill in actual paths, request/response fields, and authorization rules by reading the repository. Do not accept invented endpoint names.

14. How This Documents Vibe Coding

This prompt book documents a repeatable AI-assisted development process: natural-language requirements, repository inspection, incremental code generation and refinement, running commands, debugging actual errors, and verifying the resulting behavior.

Evidence to retain

- The actual prompts submitted to the coding assistant and relevant conversation history.
- A record of backend files created, edited, or reviewed after each prompt.
- Git commits or a change log showing the evolution of routes, middleware, services, and schema.
- Terminal output for Prisma validation, migration status, TypeScript build, tests, and server startup.
- API test results with private values removed.
- A list of completed, partial, and blocked features, including external AI or deployment dependencies.

Important: This PDF documents intended prompts and workflow; it is not independent proof that all code was AI-generated, all features are complete, the database is secure, or the application has been deployed. Use actual conversation records, version history, and test outputs when demonstrating the team's development process.